

Relatório — Atividade Mini E-commerce Distribuído

Aluno: Matheus Martins Veríssimo

1. Visão Geral da Arquitetura

O sistema é composto por três microsserviços independentes (Users :5001, Products :5002, Orders :5003), uma réplica do serviço de produtos (:5012) e um API Gateway (:8000) que atua como único ponto de entrada. Um dashboard React (:5173) consome os endpoints de monitoramento do gateway.



Toda comunica  o entre cliente e servi  os passa obrigatoriamente pelo Gateway. Cada servi  o armazena seus dados em um arquivo JSON pr prio (data/users.json, data/products.json, data/orders.json) e exp e um endpoint GET /health retornando { "status": "ok" } .

2. Comunica  o entre os Microsservi  os

A comunica  o   feita inteiramente via HTTP/REST. O API Gateway (gateway/main.py) recebe a requisicao do cliente e a repassa ao microsservi  o correto usando httpx.AsyncClient — cliente HTTP ass ncrono que n o bloqueia o event loop do FastAPI enquanto aguarda a resposta.

A comunica  o interna segue o mesmo padr o: quando o servi  o Products precisa replicar uma escrita, realiza um POST /internal/sync para a r plica (:5012) tamb m via HTTP. N o h  uso de gRPC, filas de mensagens ou qualquer protocolo al m de HTTP/REST.

3. Replica  o de Produtos e Consist ncia

Foi adotada **consist ncia forte** com rollback, implementada em products/main.py .

O fluxo de escrita ao criar um produto funciona assim:

1. Gateway encaminha POST /products para a prim ria (:5002)
2. Prim ria persiste o produto localmente
3. Prim ria chama POST /internal/sync na r plica (:5012) com timeout de 5 s
4. Se a r plica aceitar: responde 201 Created ao cliente
5. Se a r plica recusar ou estiver inacess vel: desfaz a escrita local e responde 503 ao cliente

O mesmo c digo-fonte serve as duas inst ncias; a vari vel de ambiente IS_REPLICA=true instrui a r plica a n o propagar escritas, evitando loop. O endpoint /internal/sync   idempotente — receber o mesmo produto duas vezes n o cria duplicata.

Para leituras, o gateway mant m um contador em mem ria e distribui as requisico es alternadamente entre prim ria e r plica, considerando apenas inst ncias com estado UP .

A consist ncia forte foi escolhida porque garante que qualquer leitura — independente de qual r plica responde — retorna dados v lidos. A desvantagem   que a escrita falha quando a r plica est  fora do ar. A alternativa eventual exigiria um mecanismo de reconcilia  o para detectar e resolver diverg ncias, aumentando a complexidade sem benef cio pr tico no escopo desta atividade.

4. Detec  o de Falha por Heartbeat

O heartbeat é implementado em `gateway/main.py` como `asyncio.create_task` no evento de startup — roda em paralelo com o atendimento de requisições sem bloqueá-las.

A cada 5 segundos, o gateway envia `GET /health` a cada microserviço com timeout de 2 segundos. Se um serviço falhar em 2 pings consecutivos, é marcado como `DOWN` e o evento é registrado em log com timestamp ISO 8601. Quando o serviço volta a responder, o gateway registra um evento `"recovered"` e retoma o roteamento normalmente.

Enquanto um serviço está `DOWN`, qualquer requisição destinada a ele retorna `503 Service Unavailable` imediatamente, sem tentar a conexão.

O estado atual de todos os serviços é acessível via `GET /health/status`, e o histórico de eventos via `GET /health/logs` — ambos consumidos pelo dashboard React a cada 3 segundos.

5. O que Acontece se o Serviço de Pedidos Cair

O heartbeat detecta a falha em até 10 segundos (2 falhas × 5 s de intervalo). A partir desse momento, requisições para `/orders` retornam `503` e o evento de queda é registrado. O restante do sistema — login, registro, listagem e criação de produtos — continua funcionando normalmente, pois os serviços são independentes entre si. Quando o serviço de pedidos é reiniciado, o gateway detecta a recuperação no próximo ciclo e volta a rotear normalmente.

6. Segurança com JWT

O login em `users/main.py` gera um token JWT assinado com `JWT_SECRET` (variável de ambiente, nunca versionada no repositório). O payload contém `userId`, `email`, `role` e `exp` (24 horas). Tokens sem campo `exp` são rejeitados via `options={"require": ["exp"]}`.

A proteção de rotas opera em duas camadas. No gateway, antes de repassar `POST /products`, o token é decodificado e o campo `role` é verificado — se não for `"admin"`, a requisição é barrada com `403 Forbidden` antes de chegar ao serviço de produtos. No próprio serviço de produtos, o token é revalidado novamente, garantindo proteção mesmo que o gateway seja contornado.

Senhas são armazenadas com `bcrypt.hashpw` (fator de custo 12, salt aleatório por senha) em `users/main.py`. A biblioteca `passlib` foi substituída por chamadas diretas ao `bcrypt` por incompatibilidade com `bcrypt 5.x` no Python 3.13.

Outras decisões de segurança implementadas no gateway: normalização de trailing slash com `path.rstrip("/")` para evitar bypass de rota, verificação de prefixo com boundary de segmento para evitar que `/products-internal` seja tratado como `/products`, e verificação de ownership em `GET /orders/{userId}` para impedir que um usuário acesse pedidos de outro (IDOR).

7. Dashboard de Monitoramento

O dashboard (`frontend/`) é uma aplicação React 19 + TypeScript + Vite + Tailwind CSS v4 acessível em `:5173`. Consome `GET /health/status` e `GET /health/logs` no gateway a cada 3 segundos via `setInterval`.

Cards de serviço — um card por serviço exibindo nome, porta, badge UP/DOWN com fundo verde ou vermelho, latência do último ping e horário da última verificação.

Event Stats — painel com contagem total de eventos, número de quedas e recuperações detectadas pelo heartbeat, e proporção de serviços ativos no momento.

Request Tester — painel interativo para enviar requisições ao gateway diretamente pelo navegador. Oferece atalhos para os endpoints principais, seletor de método HTTP, campo de path, campo de token de autorização e textarea de body (exibida apenas para métodos não-GET). A resposta é exibida com badge de status colorido por faixa (2xx verde, 4xx amarelo, 5xx vermelho).

Event Log — tabela unificada com todos os eventos em ordem cronológica reversa. Combina dois tipos de entrada: eventos de heartbeat vindos do gateway (marcados como `heartbeat`, com status UP/DOWN) e requisições feitas pelo Request Tester (marcadas com o método HTTP e o path chamado, com o código de resposta HTTP). Isso permite observar ao mesmo tempo o comportamento da infraestrutura e o resultado das chamadas manuais.

8. Testes Automatizados

O projeto possui 37 testes com `pytest` e `TestClient` do FastAPI, cobrindo happy path, erros de autenticação, validação de entrada e comportamento sob falha:

Serviço	Testes	Principais casos
Users	9	register, email duplicado (409), login, senha errada, JWT, 404
Products	11	CRUD, autenticação admin, rollback de réplica (503), sync interno
Orders	10	JWT, IDOR (403), quantidade inválida (422), serviço indisponível
Gateway	7	JWT ausente/inválido, admin check, trailing slash, prefixo

Para executar: `pytest <serviço>/tests/ -v` em cada diretório de serviço.

9. Limitações em Relação a um Sistema Real de Produção

Persistência: arquivos JSON não suportam consultas eficientes, transações ACID nem acesso concorrente por múltiplos processos. Um banco relacional (PostgreSQL) ou de documentos seria necessário.

Replicação: o modelo atual é síncrono e implementado na camada de aplicação. Em produção, a replicação seria gerenciada pelo próprio banco de dados (streaming replication, Raft, Paxos).

Gateway como ponto único de falha: se o gateway cair, todo o sistema fica inacessível. Produção exigiria múltiplas instâncias com load balancer na frente.

Escalabilidade horizontal: `asyncio.Lock` em memória não funciona entre processos distintos — escalar horizontalmente exigiria coordenação de locks distribuídos (Redis, etcd).

Autenticação de serviço a serviço: a comunicação interna não é autenticada entre si. Em produção, isso seria protegido por mTLS ou tokens de serviço.

Observabilidade: os logs de heartbeat são mantidos em memória e perdidos ao reiniciar o gateway. Produção exigiria persistência em sistema externo (Elasticsearch, Loki) com métricas em Prometheus/Grafana.